

RP-07 — Data Migration, Backup & Recovery Plan: Group Lesson Planner

Metadata

Field	Value
Report id	RP-07
Date	2026-08-20
Subject	Migrating existing browser-stored data into the RP-05 persistence layer, and a backup/recovery design that removes the cause of the original loss
Inputs consumed	The research agent contract; the verified-facts addendum; RP-01 §3 data model, §4 storage map and §8 defect list; RP-05 §1, §6 and §7; <code>index.html</code> in the research worktree read at 377-378, 905-975, 1187-1213, 1251-1360 and 1390-1470
Verified by me at first hand	Line ranges of the storage service (1187-1213), <code>serialize</code> (1251-1282) and <code>deserialize</code> (1283-1360) by <code>grep</code> ; the on-disk layout of Chromium <code>localStorage</code> on macOS (<code>LocalStorage/leveldb</code> , <code>LevelDB .ldb / .log / MANIFEST / CURRENT</code> files) and the WebKit <code>WebsiteData/LocalStorage/https_<host>_0.localStorage</code> naming convention, both by read-only directory listing on this Mac
Externally sourced	All Firestore backup, PITR, export, offline and quota facts; File System Access API activation and permission semantics; Chrome Sync's synced data types; Chromium profile paths; Firefox <code>localStorage</code> implementation; Time Machine and macOS erase behaviour. See §9, all fetched 2026-08-20
Inherited, not re-fetched	RP-05's storage-eviction and quota facts (MDN) and RP-01's runtime-verified defect evidence. Marked as inherited where cited
Not verified	Nothing in this report was tested against a live Firebase project, and no recovery attempt was made on any real machine. The end user's device, browser and OS are unknown
Structure note	The contract's mandatory <code>Risks</code> , <code>contradictions</code> and <code>unknowns</code> section sits unnumbered between §8 and §9 so the brief's numbered H2 order is preserved intact

Executive summary

The lost data is almost certainly gone, and no amount of engineering recovers it. If the laptop was reset with Erase All Content and Settings on Apple silicon or a T2 Mac, the erase "obliterates all the keys in effaceable storage and renders all user data cryptographically inaccessible" [S17] — that is a mathematical end, not a difficult recovery. The paths with any residual chance are, in order: a `lesson-planner-*.csv` she downloaded at some point and that still sits in a Downloads folder, an email, a phone or a synced cloud folder; a Time Machine or other external backup taken before the cleanup; another device that ever opened the app and still holds its own copy of the three keys; and partial reconstruction from the payment messages she already sent to her groups. The first and the last are more likely to

succeed than anything involving browser internals. Two hopes must be killed immediately: **Chrome Sync does not synchronise site storage** — its documented data types are bookmarks, history, passwords, payment info, settings, extensions and device metadata, with no site storage, IndexedDB or cookies [S12] — and the app makes zero network requests after load (RP-01 §6, runtime-verified), so no server anywhere holds a copy.

The cause was "one device held the only copy". Layer 1 of this design removes that cause with zero end-user discipline; the remaining manual step protects against a different risk. Once RP-05's Firestore document exists, every write lands off-device automatically, and a wiped laptop costs one sign-in. That is the structural fix, and it needs nothing from the teacher after the first sign-in.

But the cloud copy is replication, not backup, and on the free plan there is no managed backup to fall back on. Verified today: Firestore scheduled backups state "This feature requires the Blaze pricing plan" [S1]; point-in-time recovery states "Only Google Cloud projects with billing enabled can use the PITR functionality", and the window is seven days when enabled [S2]; and the managed export/import service states "Firebase projects must be on the Blaze plan" [S3]. So on Spark there is **no rollback of any kind**. A client bug, a bad import or RP-01's stray-quote CSV defect would overwrite the single cloud document and the damage would sync everywhere. This report therefore adds **Layer 2: client-rotated snapshot documents** at `users/{uid}/snapshots/{ts}` — written by the app, retained by the app, costing a handful of the 20,000 free daily writes [S20] — which restores the rollback that the free tier does not provide, still with zero user effort.

Four design decisions that follow from the app as it actually is:

1. **The machine must never choose which legacy copy is authoritative.** There is no identity and no timestamp in today's data (RP-01 §3), so no automatic rule is defensible. The import shows per-device counts and a fingerprint and requires a human to pick, and it never overwrites a non-empty target without a confirmation naming both sides.
2. **A client that meets a newer schema than it knows must refuse to write at all.** RP-05's write path is a whole-document `setDoc`, so a v1 client writing over a v2 document silently deletes every v2-only field. Read-only-and-tell-the-user is the only safe behaviour, not a nicety.
3. **"Last synced" must be driven by server acknowledgement, not by the call site.** Firestore's documented latency compensation means "your listeners will be notified with the new data *before* the data is sent to the backend" [S5]; the honest signal is `metadata.hasPendingWrites` being false, or `waitForPendingWrites()` resolving. An indicator wired to the `setDoc` call would lie in exactly the failure RP-05 says the user cannot detect.
4. **Web offline persistence is disabled by default** [S4], so the pending-write queue lives in memory: close the tab before the write reaches the server and the queued write is gone. Local `localStorage` still holds the truth, so the fix is a push-on-boot reconciliation, not trust in the queue.

Zero end-user discipline, honestly split. Achievable for the off-device copy (Layers 1 and 2). Not achievable for an independent user-held file: a page cannot write to an arbitrary filesystem path unattended. `showSaveFilePicker()` requires transient user activation and is not Baseline [S7]; `requestPermission()` needs activation too [S8]; Periodic Background Sync is experimental, not Baseline, and the browser decides the timing [S11]. The single residual manual step is one click on "Save backup" — reducible to a one-time setup on Chromium 122+, where "Allow on every visit" grants indefinite access to a stored handle [S10], but never removable in general.

One blocker inherited unchanged from RP-05. The default payment template hardcodes a real person's name, bank IBAN and tax identifier (`index.html:387–392` , plus a personal first name at `index.html:400`). Until those are removed, every artifact this report creates — the JSON backup file, the Firestore document, every snapshot — is a personal-data artifact. Nothing here may be uploaded, shared or emailed before that lands.

1. Recovery assessment for the already-lost data

The verdict, first: the data should be treated as lost. Plan the rest of the programme on that basis. What follows is not a recovery plan, it is a short list of cheap things worth trying once, ordered by the chance they pay off, plus the reasons they mostly will not.

Likelihood below is an ordinal judgement by me — **labelled inference**, not a measured probability. `Zero` means a mechanism that provably cannot hold the data. The bands are: Zero, Very low, Low, Moderate.

Path	Prerequisite	Likelihood	Act now
A previously downloaded <code>lesson-planner-*.csv</code> on any device	She used Save CSV at least once and the file survived	Moderate	Y
Payment messages she already sent to her groups	Messages still in her chat or mail history	Moderate	Y
Her own calendar, diary or notes holding the lesson schedule	She kept a schedule outside the app	Moderate	Y
A second device or browser that ever opened the app	The app was opened there and site data was not cleared	Low	Y
Time Machine or other external backup of the old profile	A backup disk existed and holds a pre-cleanup snapshot	Low	Y
Clipboard-manager history holding the last payment message	A clipboard manager was installed and retains history	Very low	Y
macOS local APFS snapshot of the old system	Same macOS install, and under 24 hours since the loss	Very low	Y
Same profile still present, only site data cleared	The browser profile directory survived the cleanup	Very low	Y
File-level undelete or carving of a deleted browser profile	Profile deleted, disk not overwritten since, no crypto-erase	Very low	Y
Mac erased with Erase All Content and Settings	Apple silicon or T2 Mac	Zero	N
Chrome Sync, Firefox Sync or iCloud syncing the data back	none — the mechanism does not carry site storage	Zero	N
A server-side copy held by the app or GitHub Pages	none — the app makes no network requests	Zero	N

Why the two Zero rows are Zero

Erase All Content and Settings. Apple documents that erasing "obliterates all the keys in effaceable storage and renders all user data cryptographically inaccessible" [S17][S18]. There is no forensic step after that. If this is what "cleaned" meant, stop here.

Browser sync. Chrome Sync's documented data types are bookmarks, history and open tabs, saved passwords, payment information, settings and preferences, extension information, and device metadata. Website `localStorage`, site data, IndexedDB and cookies are not in the list [S12]. A signed-in Chrome on a new laptop restores her bookmarks and gives back nothing of this app's data. Expect her to have already assumed otherwise.

Where each browser physically keeps `localStorage`

This matters only for the two paths that involve reading bytes off a disk: a surviving profile whose site data was cleared, and a restored backup of a deleted profile.

Engine	Location	Format	How to confirm
Chrome, Edge, Brave, Chromium	<code><user data dir>/<Profile>/Local Storage/leveldb/</code> [S13]	LevelDB directory: <code>*.ldb</code> , <code>*.log</code> , <code>MANIFEST-*</code> , <code>CURRENT</code> , <code>LOCK</code> , <code>LOG</code>	Verified by me on macOS by listing that directory
Chromium user data dir, macOS	<code>~/Library/Application Support/Google/Chrome</code> [S13]	container for per-profile folders, Default being the first	Documented in Chromium's own <code>user_data_dir.md</code> [S13]
Chromium user data dir, Windows	<code>%LOCALAPPDATA%\Google\Chrome\User Data</code> [S13]	same	same
Firefox, current	per-origin database file under the profile's <code>storage/default/</code> tree [S14]	SQLite	<code>find <profile> -path '*storage/default*' -name '*.sqlite'</code>
Firefox, legacy store	<code><profile>/webappsstore.sqlite</code> — Mozilla describes it as "a single database file containing all origins' data" [S14]	SQLite	open read-only and query for the origin
Safari and other WebKit apps, macOS	<code>.../WebKit/WebsiteData/LocalStorage/https_<host>_0.localstorage</code>	SQLite, one file per origin	Verified by me: that naming convention exists on this Mac

Two practical notes. Chromium's store is a log-structured LevelDB, so a value removed by "clear site data" can survive inside older `.ldb` or `.log` segments until compaction rewrites them — which is the one case where a `strings`-style scan of a surviving profile is genuinely worth ten minutes. And the exact Safari container path varies by macOS release: Apple does not document it, so do not type a path from memory. Locate it instead.

```
# macOS: locate any WebKit localStorage file for the app's origin, read-only.
find ~/Library -name 'https_denyslstopadskyy.github.io_0.localstorage' 2>/dev/null

# Chromium family: scan a surviving profile's LevelDB for the app's keys without opening
the DB.
cd "$HOME/Library/Application Support/Google/Chrome/Default/Local Storage/leveldb"
LC_ALL=C grep -a -o -m 5 'groupLessonPlannerData.\{0,400\}' ./*.ldb ./*.log 2>/dev/null

# Anywhere: find a previously downloaded export by the app's own filename pattern.
mdfind -name 'lesson-planner-' 2>/dev/null
find / -name 'lesson-planner-*.csv' 2>/dev/null
```

Time-sensitivity: two different clocks

They are often conflated, and they expire at very different speeds.

Clock	Documented behaviour	What it means here
macOS local snapshots	Time Machine "saves one snapshot of your startup disk approximately every hour, and keeps it for 24 hours", plus one snapshot of the last successful backup until space is needed, and deletes them "as they age or as space is needed" [S16]	If more than about a day passed, this path is already closed. It is the fastest-expiring option
Time Machine on an external disk	"hourly backups for the past 24 hours, daily backups for the past month, and weekly backups for all previous months", with "the oldest backups are deleted when your backup disk is full" [S15]	Older losses survive only at weekly granularity, and continued backups eventually age the pre-incident snapshot out. Disconnect the backup disk or stop backups until it has been checked
Unallocated blocks on the disk	Not vendor-documented. Continued use of a machine overwrites free space; SSD trim makes deleted-file recovery unreliable in general	Inference , standard practice: if undelete is going to be attempted at all, stop using that disk now. This clock does not affect an existing snapshot — those are separate

If a recovery attempt is made at all, do these first, in this order

1. **Stop using the affected machine** for anything that writes to disk. Do not install a recovery tool onto it.
2. **Do not let a fresh Time Machine backup run** until the existing history has been inspected. Disconnect the backup disk.
3. **Search every other device** — second laptop, phone, tablet, work computer — for `lesson-planner-*.csv` and for the app's origin in browser site data. Any device that ever opened the app has, or had, its own complete copy.

4. **Search mail and chat** for a sent `.csv` attachment and for the payment messages themselves. The messages contain the month, the lesson count and the total per group, which is enough to reconstruct the money even if the individual dates are gone.
5. **On any device where the app still has data, dump it immediately** — before touching the app, before any update, before any import. Quick win QW-1 is exactly this and needs no code change.
6. Only then consider disk-level recovery, and only if steps 1 to 5 came back empty.

Steps 3 to 5 are the ones that actually pay. And the honest framing for the conversation with the teacher: the goal is not to get August back, it is to make sure this is the last time a loss like this is possible.

2. Target schema definition and versioning rules

The canonical backup envelope

One shape serves both sinks — the file the user holds and the document in the cloud. The three existing key names are used verbatim as the member names inside `data`, so the mapping between file and browser is auditable by eye and no rename is introduced anywhere.

```
{
  "format": "group-lesson-planner-backup",
  "schemaVersion": 1,
  "exportedAt": "2026-08-20T17:04:11.238Z",
  "source": {
    "origin": "https://denyslystopadskyy.github.io",
    "device": "8f2c1a44-3b7e-4c90-9d21-6ae0f5b1c7d2",
    "label": "laptop-chrome"
  },
  "counts": {
    "groups": 2,
    "dates": 41,
    "overrideDates": 41,
    "monthKeys": 6,
    "templateChars": 446
  },
  "fingerprint": "fnv1a32:9f2b41c7",
  "data": {
    "groupLessonPlannerData": [
      {
        "name": "Beginners A2",
        "price": 200,
        "currency": "PLN",
        "dates": ["2026-08-04", "2026-08-11"],
        "monthlyOverrides": {
          "2026-08": { "price": 77, "dates": ["2026-08-04", "2026-08-11"] }
        }
      }
    ],
    "groupLessonPlannerSettings": { "defaultCurrency": "PLN" },
    "paymentTemplate": "<the teacher's own template string, or null when the key is absent>"
  }
}
```


Notes on the fields, each tied to something RP-01 established. `data.groupLessonPlannerData` is byte-identical to what the key holds today (RP-01 §3), so import and export are copies, not transformations. `data.paymentTemplate` is a raw string because that is what `index.html:1208` and `index.html:1211` read and write — not JSON — and `null` distinguishes "key absent" from "empty string", which matters because RP-01 D9 showed an invisible control can write an empty string to that key. `counts.dates` and `counts.overrideDates` are both present precisely so a divergence in RP-01's denormalised `dates` invariant is visible in the envelope rather than silently copied. `source.device` is a per-device UUID the app generates once and keeps locally; it exists so the multi-device preview in §3 and the "last synced" line in §4 can name a device instead of guessing.

The Firestore document

RP-05 §7 fixed the shape; this report adds only the three fields that migration, conflict handling and verification need, and changes nothing else.

```
{
  "schemaVersion": 1,
  "groups": [],
  "settings": { "defaultCurrency": "PLN" },
  "paymentTemplate": "<the teacher's own template string>",
  "clientUpdatedAt": "<epoch milliseconds from Date.now(), stored as a number>",
  "serverUpdatedAt": "<Firestore serverTimestamp()>",
  "writerDevice": "8f2c1a44-3b7e-4c90-9d21-6ae0f5b1c7d2",
  "fingerprint": "f9v1a32:9f2b41c7"
}
```

`writerDevice` and `fingerprint` are additive. `serverUpdatedAt` is promoted from RP-05's "diagnostics only" to a real guard — see §4 on clock skew.

Local schema-version storage

Item	Value	Rationale
New key	<code>groupLessonPlannerSchemaVersion</code>	Additive fourth key. Keeps the three existing keys' shapes untouched, which every RP-01 and RP-05 quick win depends on
Contents	a bare decimal integer as a string, e.g. <code>1</code>	Not JSON, so a parse failure is impossible. Matches the existing precedent of <code>paymentTemplate</code> holding a raw string
Absent	means version 0 — the legacy, unversioned data every current user has	No migration needed to <i>start</i> versioning: absence is the version
Snapshot keys	<code><key>.pre-v<N>.<epoch></code>	Written before any migration or destructive replace, so every one-way step is reversible
Corrupt keys	<code><key>.corrupt.backup</code>	Already specified by RP-05 quick win 1. Reused, not redefined

Versioning rules

1. **schemaVersion** is a single monotonically increasing integer. No semver, no per-entity versions. One writer, one document, one number. Anything more is disproportionate at this scale (RP-05 §5).
2. **Migrations are an ordered list of pure functions**, `m1 : v0 -> v1`, `m2 : v1 -> v2`, each taking and returning the `data` object of the envelope and touching nothing else — no DOM, no `localStorage`, no network. This makes them unit-testable, which matters because a migration bug is a data-loss bug.
3. **A record older than the running client** is migrated forward on load: snapshot all keys to `<key>.pre-v<N>.<epoch>`, apply every migration in sequence, validate the result, then write back and set the version key. If any migration throws, **write nothing at all** and go read-only with a message naming the version found and the version expected. A partial migration is the one outcome that must be impossible.
4. **A record newer than the running client** — stored version greater than the constant compiled into the running app — puts the client into read-only mode. It must not write to `localStorage`, must not `setDoc`, and must not "helpfully" strip fields it does not recognise. It should render what it can understand, show a plain message ("This data was saved by a newer version of the planner. Reload the page to update."), and disable every editing control. The reason is mechanical, not stylistic: RP-05's write path is a whole-document `setDoc`, so one write from a v1 client silently deletes every field a v2 client added. There is no down-migration and there is no undo on the Spark plan (§5). Two devices make this real the first time one of them loads a cached older page.
5. **An import file newer than the client is refused, not truncated**. Same reasoning, same message, no partial apply.
6. **Migrations are never written to accept malformed input silently**. RP-01 D4 proved the app can persist a month key like `5-08-10`. A migration that encounters one must report it and continue with the rest of the data, never abort the whole run — the opposite of the CSV importer's behaviour at `index.html:1337`, where one bad month kills the entire restore.
7. **The three existing key names never change**. If a rename is ever wanted, it is a migration of its own: write the new key, verify, keep the old key untouched for one release, then delete it in the following release. Not in the same step.

3. One-time import design, as executable steps

The frame that keeps this simple: **one transform, two sinks**. Legacy `localStorage` becomes a canonical envelope (§2); the envelope is then written to a file (phase 1, ships first per RP-05 §6) or to Firestore (phase 2). Dry-run, idempotency and reversibility all fall out of that split, and the same importer stays valid across both phases.

Step 0 — Prerequisites

- The personal-data removal at `index.html:387-392` and `index.html:400` has landed (RP-05 §7 blocker). Until it has, an envelope file must not be emailed, uploaded or placed in a shared folder, because `data.paymentTemplate` will contain a real IBAN and tax identifier for any user who never edited the template.
- The storage layer is guarded (RP-05 quick win 1), so a corrupt value cannot kill `init()` mid-import.

Step 1 — Collect, on every device, before anything else

Run the raw dump (QW-1) on every device and browser that might hold data. It reads the three keys as **raw strings and does not parse them**, so it works even when the stored JSON is corrupt — the case where a parsing exporter would fail on exactly the data you most need. Name each file after the device.

Step 2 — Normalise to the envelope

For each dump: parse what parses, count what counts, compute the fingerprint (§7), and emit the envelope with `source.label` set to the device name. Anything unparseable is carried through verbatim in a `problems` list rather than dropped.

Step 3 — Dry run and preview, which is where the human decides

This step is a developer-run procedure, not app UI, and it is deliberately not a quick win. Its input is the envelope files from Step 2; its home is the maintainer's dev console or a throwaway local page, never `index.html`. Building a multi-copy comparison and merge tool into the deployed app would be disproportionate for an operation that happens once. The human doing the adjudicating is therefore **the maintainer, sitting with the teacher** — she supplies the knowledge of which device she last taught from, he reads the counts. Nothing in this section is written for her to do alone, unlike §6.

Show one row per collected copy, and nothing else:

Copy	Groups	Dates	Months	Template	Fingerprint
laptop-chrome	4	96	11	present, 512 chars	fnn1a32:9f2b41c7
tablet-safari	2	18	3	absent	fnn1a32:41ca0b19

Plus, per copy, the problems found: malformed month or date keys (RP-01 D4), a currency that is not exactly three letters (RP-01 D5, which makes a group permanently unopenable), duplicate group names (which the CSV importer at `index.html:1309` and `index.html:1323` would silently merge), and any divergence between `dates` and `overrideDates`.

Step 4 — Which copy is authoritative: the machine must not decide

The app has no user identity and no timestamp anywhere in its persisted data (RP-01 §3), so **there is no field that could support an automatic rule**. "Most groups wins" loses a device where she deleted a finished course. "Newest file wins" measures when the dump was taken, not when the data was written. The defensible design is therefore:

- The preview presents the counts and marks the largest copy as a **hint**, explicitly labelled as a hint.
- A human picks. One click, with both fingerprints on screen.
- If each copy contains groups the other lacks — checked by group name — the tool says so and offers a **merge by group name**, listing every name that exists on one side only, and refusing to merge two groups that share a name. It never merges dates inside a single group across copies; that is a genuine conflict and it is presented as one.

- From phase 2 onward the question mostly disappears: `clientUpdatedAt` and `serverUpdatedAt` exist, so §4's rule applies and only legacy copies need adjudication.

Step 5 — Snapshot the target, then write

1. Snapshot all three keys of the **target** to `<key>.pre-import.<epoch>` — even when the target looks empty. Cheap, and it is the undo.
2. If the target is non-empty, require a confirmation that names both sides: "Replace 2 groups / 18 lessons on this device with 4 groups / 96 lessons from laptop-chrome?" Never a bare "Are you sure?". RP-01 D11 verified that today's importer replaces everything with no dialog at all, and RP-01 D12 verified it can do so with garbage.
3. Write the three keys, then `groupLessonPlannerSchemaVersion`.
4. Re-render, then recompute the fingerprint from what is now in storage and compare it with the envelope's. Mismatch means stop and restore from the snapshot.

Step 6 — Idempotency

Write a marker key `groupLessonPlannerImportedFrom` holding `{ fingerprint, importedAt, label }`. If an import is attempted whose envelope fingerprint equals the stored marker's fingerprint **and** the current data fingerprint still equals it, the import is a no-op and says so. Re-running the same import can therefore never duplicate or clobber, which matters because a stressed user will click twice.

Step 7 — Phase 2, into Firestore

The same envelope, one `setDoc` to `users/{uid}`, with the guard the other direction: if the remote document already exists and its `fingerprint` differs from the local one, do not write — run Step 3's preview with the remote copy as one of the rows and let the human choose. First sign-in on a device that already has local data is exactly this case, and it is the most likely moment for an accidental overwrite in the whole design.

Step 8 — Reversal

`restoreSnapshot(epoch)` copies `<key>.pre-import.<epoch>` back over the three keys and reloads. Snapshot keys are never cleared by `clear()` (`index.html:1203–1206` removes exactly two keys) and are pruned only by an explicit maintenance action, keeping the most recent three.

4. Conflict and offline behaviour specification

These are requirements, not options. Each row states what the app must do.

Scenario	Required behaviour
Local write, any state	Write the three <code>localStorage</code> keys synchronously first, exactly as <code>save()</code> does at <code>index.html:1199–1202</code> . The local write is the commit; everything else is replication

Scenario	Required behaviour
Remote write	Debounced write-behind, whole document, coalesced over 1 to 2 seconds of quiet (RP-05 §7). Never on keystroke
Two devices used at different times	Compare <code>serverUpdatedAt</code> first; if either side lacks it, compare <code>clientUpdatedAt</code> . Greater wins, whole document. No field-level merge
Two devices, both with unsynced local edits	The later-arriving write wins and the earlier device's unsynced edits are lost. This is accepted (see §8) and must be <i>disclosed</i> by the sync indicator, which will show the losing device as unsynced
Two tabs on one device	Last write wins per Firestore's documented behaviour for multiple changes to the same document [S4]. Both tabs must reload state on <code>visibilitychange</code> becoming visible so the stale tab stops showing stale data
Offline or network error	Silent. The local write already succeeded. Retry on the next mutation and on the next load. Never a modal (RP-05 §7)
Tab closed with a write still pending	Treated as a normal unsynced state, not an error. Web offline persistence is disabled by default [S4], so an in-memory queue can be lost; the app must therefore compare fingerprints on every boot and push if local differs from remote, rather than trusting the SDK queue
Write appears to succeed but never reaches the server	The indicator must not advance. It advances only when <code>metadata.hasPendingWrites</code> is false for the document, or <code>waitForPendingWrites()</code> resolves — because local writes notify listeners "before the data is sent to the backend" [S5]
Not signed in	The app works fully, local-only, with a quiet "Sign in to back up" affordance. Not an error state (RP-05 §7)
<code>permission-denied</code>	Persistent non-blocking banner, and log it. Means the rules or the UID are wrong; needs the maintainer (RP-05 §7)
Remote document malformed or fails validation	Reject it, keep local, never assign it into state. Validate <code>schemaVersion</code> and that <code>groups</code> is an array first (RP-05 §7, RP-01 D14)
Remote <code>schemaVersion</code> greater than the client knows	Read-only mode, no writes of any kind, message telling the user to reload. See §2 rule 4
Device clock is wrong	<code>clientUpdatedAt</code> alone must never decide. If the last acknowledged <code>serverUpdatedAt</code> and the local <code>clientUpdatedAt</code> disagree by more than an hour, show a "device clock looks wrong" notice and prefer <code>serverUpdatedAt</code> ordering for every subsequent comparison
<code>QuotaExceededError</code> from <code>localStorage</code>	Catch, warn, keep in-memory state. Unguarded today (RP-01 D14)

Scenario	Required behaviour
Explicit sign-out	Warn that the device will stop backing up, and require confirmation. Firebase's default <code>local</code> persistence is cleared only by explicit sign-out (RP-05 §6), so this is the one click that silently returns her to the pre-incident arrangement

The sync indicator, specified

RP-05 calls this mandatory rather than cosmetic. Making it truthful is a design constraint, not a label choice.

State	Condition	Displayed text	Treatment
Local only	not signed in	Saved on this device only	neutral, with a sign-in link
Pending	signed in, <code>hasPendingWrites</code> true	Saving...	neutral
Synced	<code>hasPendingWrites</code> false and no error	Backed up HH:MM	neutral
Stale	signed in, last ack older than 3 days while local edits exist	Not backed up for N days	warning colour
Broken	last ack older than 14 days while local edits exist, or a persistent error	Backup is not working – tell the developer	warning colour, persistent

The 14-day threshold is RP-05's switching trigger, reused deliberately so the user-visible signal and the maintainer's escalation rule are the same number. The 3-day warning exists because a non-technical user needs to see a problem while it is still one week of lessons and not one term.

5. Backup design

Four layers, and only two of them are backups

Layer	What it is	Cadence	Where	Retained	User effort
0	<code>localStorage</code> cache	every mutation	her browser profile	until wiped	none
1	Firestore <code>users/{uid}</code>	debounced, per change	Google, <code>europe-central2</code>	current version only	none after first sign-in
2	Snapshot docs <code>users/{uid}/snapshots/{ts}</code>	first write of each calendar day	same project	last 14 daily plus first-of-month for 12 months	none
3	JSON envelope file	on click, or per session on Chromium	wherever she saves it	as many as she keeps	one click

Layer 0 is not a backup: it is the thing that was lost. **Layer 1 is not a backup either** — it is a single mutable replica. It removes the incident's root cause (one device, one copy) and it protects against nothing else: overwrite the document with garbage and the garbage is what you have, on every device.

That gap is not covered by the provider on the free plan, and this is the report's most consequential verified finding:

Managed recovery mechanism	Requirement	Available on Spark
Scheduled backups, up to 14 weeks retention	"This feature requires the Blaze pricing plan" [S1]	No
Point-in-time recovery, seven-day window when enabled	"Only Google Cloud projects with billing enabled can use the PITR functionality" [S2]	No
Managed export to a Cloud Storage bucket	"Firebase projects must be on the Blaze plan to use the managed export and import service" [S3]	No

Note also that even where they are available, scheduled backups are not files you download: a restore "creates a new database" rather than replacing the original [S1]. So Layer 2 is not a workaround for a missing convenience — it is the only rollback that exists at this budget, and it must be built in the client.

Layer 2 in detail

- **Trigger:** on the first successful sync of each calendar day, copy the just-written document into `users/{uid}/snapshots/{YYYY-MM-DDTHH-mm-ssZ}` .
- **Retention, enforced by the client:** keep the newest 14 daily snapshots plus the first snapshot of each of the last 12 months; delete the rest in the same pass. Deterministic, inspectable, no console configuration. Firestore TTL policies are an alternative but delete only "typically within 24 hours after its expiration date" [S6], which is looser than a rule the app can enforce exactly.
- **Cost:** at most two writes and a few deletes per active day against 20,000 free writes and 20,000 free deletes per day [S20], and about 14 KB per snapshot (RP-01 §3 inference) against 1 GiB of free storage [S20]. Twenty-six retained snapshots at that size is about 0.03% of the 1 GiB allowance. Capacity is not a consideration.
- **Why it earns its place:** it is the mitigation for RP-05's otherwise-unmitigated "sync propagates corruption" risk, and it needs nothing from the user.

Layer 3, and the precise limits of automation

The format is the §2 envelope: human-readable JSON, re-importable by the same code path that reads it, and containing all three keys — which the current CSV export does not, since it omits the template entirely (RP-01 D16).

What a browser page can and cannot do here, verified:

Capability	Documented position	Consequence
Save a file the user chooses a location for	<code>showSaveFilePicker()</code> requires transient user activation and throws <code>SecurityError</code> without it; it is "not Baseline because it does not work in some of the most widely-used browsers" [S7]	One click minimum, and Chromium-only
Re-use a previously chosen file in a later session	Handles "can be serialized into an IndexedDB database instance" [S9], but <code>requestPermission()</code> also requires transient user activation [S8]	Storing the handle removes the file picker, not the gesture — unless the grant is persistent
Persistent file permission	Chrome 122+ offers "Allow on every visit", giving "indefinite access unless access is revoked"; installed apps persist permissions automatically [S10]	With a persistent grant, <code>queryPermission()</code> returns granted, so no <code>requestPermission()</code> and hence no gesture is needed — inference from the two documented facts above , worth testing before relying on it
Run on a schedule with the tab closed	Periodic Background Sync is experimental and not Baseline, needs a service worker, and "the user agent might also take into account other factors which affect when the service worker receives the event" [S11]	No dependable unattended backup, in any browser
Write to an arbitrary filesystem path unattended	Not possible in any engine	The classic "just write it to Dropbox every night" idea does not exist on the web

The honest verdict on zero end-user discipline

Split it in two, because the answer differs.

- **The failure that actually happened — one device holding the only copy — is fixed with zero discipline.** Layers 1 and 2 write off-device automatically on every change, and the user's only act is one sign-in, once. Nothing depends on her remembering anything after that.
- **An independent copy she controls still costs one click.** The residual manual step is: press "Save backup" and keep the file. On Chromium 122+ it reduces to a one-time setup, after which the app can rewrite the same file whenever it is open [S10]. On Safari and Firefox it stays one click per backup, because the picker API is not available there [S7].

The mitigations for that residual step are the ones RP-05 already specified: the "Last backup: N days ago" indicator with a warning past the threshold, and the fact that Layers 1 and 2 mean a missed Layer 3 backup is no longer a single point of failure. That is a genuinely different posture from the one that failed, and it should be described to the teacher in exactly those terms: *the app now backs itself up; the file is your escape hatch if the developer disappears.*

Where backups land, and the personal-data constraint

Destination	Suitable	Why
Her own Downloads folder or Documents	Yes, after the blocker clears	On her disk, in Poland, no third party
A consumer sync folder she already uses	Yes, after the blocker clears	Gets the file off the device too. Adds a third party to the data path, which is a conscious choice, not a default
Email to herself	No	The envelope carries <code>paymentTemplate</code> — a personal IBAN and tax identifier until <code>index.html:387–392</code> is fixed — into a mail provider and its backups
Any shared folder or repository	No	Same reason, permanently
The research repository	Never	It is public

6. Restore procedure

Written to be followed by a non-technical person under stress. Read all of Procedure A before doing anything; you cannot make things worse by reading.

None of this is executable today, and the prerequisites differ per procedure. Procedure A requires phase 2 — the Firestore document and the sign-in of RP-05 §7 — plus the sync indicator specified in §4. Procedure B requires the Save Backup and Load Backup controls of RP-05 quick win 2, the import confirmation of RP-05 quick win 5, and the both-sides wording of §3 step 5.2. Procedure C requires phase 2 and the snapshot layer of §5. Until each lands, the corresponding procedure is a specification for what must be buildable, not an instruction sheet to hand over. Do not give the teacher Procedure B while the buttons it names do not exist — that is the prior-art failure RP-01 §8 documented, repeated.

Procedure A — restore from the cloud (the normal case, after phase 2)

1. Open the planner on the device you want to restore to:
`https://denyslystopadskyy.github.io/lesson-planner/`.
2. **Do not add any group and do not press any button yet.** If you add data first, you make the next steps harder.
3. Press **Sign in** and sign in with the same Google account you used before. If you are unsure which account, stop and ask the developer — signing in with the wrong one shows an empty planner and that is frightening for no reason.
4. Wait until the line at the top says **Backed up** with a time. That means your data has arrived.
5. Check three things, in this order: the number of groups shown; the lesson count on one group you remember; and the total for the current month inside that group.
6. If all three look right, you are done.

7. If the planner is empty or the numbers are wrong, **stop. Do not add anything, do not import anything, do not press Clear All Data.** Take a screenshot and contact the developer. Anything you type now can overwrite a good copy that is still up there.

Procedure B — restore from a backup file

1. Find your backup file. It is a `.json` file whose name starts with `lesson-planner-backup`. Pick the newest one you trust.
2. Open the planner.
3. **Before importing, make a safety copy of what is there now:** press **Save backup** and keep that file. Even if the current data looks wrong, keep it.
4. Press **Load backup** and choose your file.
5. Read the dialog. It will say how many groups will be replaced and how many are coming in — for example, replacing 2 groups with 4 groups. If those numbers are not what you expect, press **Cancel**. Cancelling changes nothing.
6. If the numbers look right, confirm.
7. Verify exactly as in Procedure A step 5: group count, one group's lesson count, one month's total. Then open **Edit Template** and confirm your payment text is the one you wrote.
8. If anything is wrong, press **Load backup** again and choose the safety copy from step 3. That puts you back exactly where you started.

Procedure C — developer restore from a snapshot document

1. In the Firebase console, open Firestore and the collection `users/{uid}/snapshots`.
2. Sort by document id — the id is the ISO timestamp — and pick the newest snapshot **before** the damage.
3. Copy its fields to a local file first. Never edit in place.
4. Verify the counts and fingerprint against §7 before writing anything.
5. Write the snapshot's fields to `users/{uid}`, setting `clientUpdatedAt` to the current time so every device pulls it, and set `writerDevice` to a value identifying the manual restore.
6. Have the user reload on one device and run the §7 verification with her.
7. Record what happened, in the repository, with the snapshot id used. A restore nobody wrote down is a restore that cannot be audited next time.

7. Post-migration verification procedure and acceptance criteria

What is compared

Check	Method	Passes when
Group count	length of <code>groupLessonPlannerData</code>	identical before and after
Total date count	sum of <code>group.dates.length</code>	identical

Check	Method	Passes when
Override date count	sum of every <code>monthlyOverrides[m].dates.length</code>	identical, and equal to the total date count
Distinct month keys	union of <code>Object.keys(monthlyOverrides)</code>	identical
Per-month prices	every <code>monthlyOverrides[m].price</code>	identical
Template	<code>paymentTemplate</code> character length and exact string	identical, with absent distinguished from empty
Fingerprint	canonical checksum below	identical

The fingerprint, defined precisely

`JSON.stringify` is insertion-ordered, so a naive checksum of two semantically identical objects can differ. The canonical form therefore sorts object keys and sorts array members, which makes the checksum insensitive to key order and to the order of a date list — neither of which carries meaning in this model. It is a comparison artifact only and is **never written back to storage**.

One consequence must be stated rather than discovered later: because the recursion sorts every array, it sorts the `groups` array too, so **the fingerprint proves set-equality of content, not positional equality**. A migration that reorders groups produces an identical fingerprint. That is the right trade — RP-01 §3 records that group identity is the array index via `App.state.editingIndex`, but that index is transient, never persisted, and the card list is re-sorted by name on every render (`index.html:1029-1033`), so position is not part of the persisted contract. Anything that ever *does* persist an index must add a positional check of its own.

```
// Data fingerprint. Paste into DevTools on the app's origin, or expose as
App.utils.dataFingerprint().
function canonical(value) {
  if (Array.isArray(value)) {
    return '[' + value.map(canonical).sort().join(',') + ']';
  }
  if (value !== null && typeof value === 'object') {
    return '{' + Object.keys(value).sort()
      .map(k => JSON.stringify(k) + ':' + canonical(value[k])).join(',') + '}';
  }
  return JSON.stringify(value === undefined ? null : value);
}
function fnv1a32(text) {
  let h = 0x811c9dc5; // FNV-1a 32-bit offset basis
  for (let i = 0; i < text.length; i++) {
    h ^= text.charCodeAt(i);
    h = Math.imul(h, 0x1000193); // FNV-1a 32-bit prime
  }
  return (h >>> 0).toString(16).padStart(8, '0');
}
function dataFingerprint() {
  const groups = JSON.parse(localStorage.getItem('groupLessonPlannerData') ?? '[]');
  const settings = JSON.parse(localStorage.getItem('groupLessonPlannerSettings') ?? '{}');
```

```

const template = localStorage.getItem('paymentTemplate');
const monthKeys = new Set();
let dates = 0;
let overrideDates = 0;
const problems = [];
groups.forEach(g => {
  dates += (g.dates ?? []).length;
  (g.dates ?? []).forEach(d => {
    if (!/^\d{4}-\d{2}-\d{2}$/.test(d)) problems.push('bad date: ' + d);
  });
  const currency = typeof g.currency === 'string' ? g.currency : '';
  if (!/^[A-Za-z]{3}$/.test(currency)) {
    problems.push('bad currency on group: ' + g.name);
  }
  Object.keys(g.monthlyOverrides ?? {}).forEach(m => {
    monthKeys.add(m);
    overrideDates += ((g.monthlyOverrides[m] ?? {}).dates ?? []).length;
    if (!/^\d{4}-\d{2}$/.test(m)) problems.push('bad month key: ' + m);
  });
});
return {
  groups: groups.length,
  dates,
  overrideDates,
  datesParity: dates === overrideDates,
  monthKeys: monthKeys.size,
  templateChars: template === null ? null : template.length,
  problems,
  fingerprint: 'fnv1a32:' + fnv1a32(canonical({ groups, settings, template })),
};
}
console.log(JSON.stringify(dataFingerprint(), null, 2));

```

Procedure

1. **Before** the migration or restore, run the snippet on the source and save its output.
2. Perform the migration or restore.
3. Run the snippet again on the target.
4. Compare every field. `problems` may **shrink** — a migration is allowed to repair a malformed month key or a bad currency — but every repair must be listed in the migration's report, and the group, date, month and override counts must be unchanged by any repair.
5. Spot-check three things in the running UI, chosen because they exercise different code paths: one group card's `{n} planned lessons count (render.groups, index.html:1045)`; one month row's `Total: figure` (the override arithmetic at `index.html:1086`); and the generated payment message for that month, checked for correct month name, lesson count and total (`index.html:1367–1378`).
6. Open **Edit Template** and confirm the template is the user's own, not the default.
7. Record the before and after outputs in the repository next to the migration code.

Acceptance criterion

The migration is accepted only when, for every migrated copy: `groups`, `dates`, `overrideDates`, `monthKeys` and `templateChars` are identical before and after; `datesParity` is true after; the `fingerprint` is identical unless a repair was applied, in which case every difference is explained by a listed repair and the five counts are still identical; the three UI spot checks in step 5 match the values recorded before; and a snapshot key `<key>.pre-import.<epoch>` exists for all three keys. Any single failure means restore from the snapshot and do not proceed.

Note the parity check deserves its own attention. `datesParity` being false is not caused by migration — it is RP-01's denormalisation defect surfacing (`group.dates` and the override arrays are synchronised only in `saveDateChanges` at `index.html:745–772`). A faithful copy preserves the divergence silently. Migration is the natural moment to *detect* it, report it, and let the maintainer decide which side is right; it is not the moment to silently pick one.

8. Residual failure modes

Every way data can still be lost after this plan is implemented. "Accepted" means the residual risk is knowingly carried, not that it is harmless.

Failure	Mitigation	Accepted
Corrupt JSON in <code>localStorage</code> produces a dead page (RP-01 D14)	Per-key try/catch in <code>load()</code> , raw value copied to <code><key>.corrupt.backup</code> , non-blocking message, <code>init()</code> always completes (RP-05 quick win 1)	N
CSV import silently replaces everything (RP-01 D11)	Confirmation naming both counts (RP-05 quick win 5) plus a <code>pre-import</code> snapshot and an undo (§3 steps 5 and 8, QW-2)	N
CSV export omits the payment template (RP-01 D16)	The JSON envelope carries all three keys; the CSV path is demoted to an interchange format and must never be described as a backup	N
Malformed month key from the year input aborts a re-import (RP-01 D4)	Constrain the year input (RP-01 quick win 3); the importer reports bad keys per record and continues instead of aborting (§2 rule 6)	N
A balanced stray quote in a CSV silently destroys data (RP-01 D12)	Same confirmation, snapshot and undo as above. The CSV parser at <code>index.html:1413–1455</code> is not made stricter; the JSON path is the one that gets trusted	Y — the CSV parser's leniency is left in place deliberately
A three-letter but wrong currency, or a non-letter currency, makes a group permanently unopenable (RP-01 D5)	Importer quarantines and repairs the currency before writing (QW-5). A currency that is three letters but meaningless still formats without error	Y for semantically wrong but well-formed codes

Failure	Mitigation	Accepted
A v1 client overwrites a v2 document and strips fields	Read-only refusal on a higher <code>schemaVersion</code> (§2 rule 4). This is the only defence, because <code>setDoc</code> writes whole documents	N
A client bug or bad import writes garbage that syncs everywhere	Layer 2 snapshot documents give point-in-time rollback the free tier does not (§5). Restore is Procedure C	N
Two devices with unsynced edits — the later write wins and the earlier edits are lost	The sync indicator shows the losing device as unsynced. No field-level merge is built: one writer, and a merge engine is disproportionate (RP-05 §5)	Y
Silent write failure the user cannot detect	Indicator driven by <code>hasPendingWrites</code> and <code>waitForPendingWrites</code> , never by the call site (§4)	N
Tab closed with a queued write, in-memory queue lost	Fingerprint comparison and push on every boot (§4); web persistence is off by default [S4]	N
Explicit sign-out returns her to a single-device copy	Confirmation dialog naming the consequence (§4)	Y — she is allowed to sign out
Google account lost, blocked by an employer admin, or the project deleted	Layer 3 file is independent of the provider; RP-05's switching triggers apply	Y
Firebase changes the Spark terms, or requires a payment method	RP-05's switching triggers; the local path has no dependency to rot	Y
The whole Firebase project is deleted by mistake	Layer 3 file only. There is no managed backup on Spark [S1][S3]	Y — the reason Layer 3 exists
She never presses "Save backup", so no independent copy exists	Staleness indicator and nag (RP-05 quick win 3); Layers 1 and 2 mean this is no longer a single point of failure	Y — this is the residual manual step named in §5
Stored XSS from a group name reads or rewrites the synced document (RP-01 D1)	Fix D1 before or with the sync work (RP-05 §8). A signed-in session in a page with an injection sink is a data-exfiltration path, not only a defacement	N
The backup file itself leaks a personal IBAN and tax identifier	Blocker: remove <code>index.html:387–392</code> and <code>index.html:400</code> first. Until then, no upload, no email, no shared folder (§5)	N
WebKit's seven-day script-storage sweep clears the Firebase session on Safari	She signs in again and the data returns automatically (RP-05 §6). Only the local-only fallback is materially harmed	Y

Failure	Mitigation	Accepted
Browser profile wiped again	Layer 1 makes this a one-sign-in event instead of a total loss. This is the incident's root cause, and it is closed	N
Every device lost at once, with no Layer 3 file and the project intact	Sign in on a new device. Covered	N
Every device lost, the account inaccessible, and no Layer 3 file kept	Nothing. Total loss	Y — the irreducible residue, and the reason to keep one file

Risks, contradictions and unknowns

Contradictions found

None between primary sources in this report's own scope. The three Firestore recovery pages agree with each other and with the pricing page: scheduled backups need Blaze [S1], PITR needs billing [S2], managed export needs Blaze [S3], and Spark needs no payment method [S20]. The docs-versus-changelog contradiction that this report's brief attributed to Firestore's point-in-time-recovery window is in fact **Supabase's paused-project restore window** in RP-05 §8, and the "backups not downloadable" finding is Supabase's Free Plan. See "Where an input was wrong" below.

One near-contradiction worth recording rather than resolving: Firestore's scheduled-backup docs offer retention "up to 14 weeks" [S1] while PITR offers seven days [S2]. These are different products, not a conflict, and neither is reachable without billing.

Everything marked TBD

TBD	Why unresolved	What resolves it
What "the laptop was cleaned" actually means	Not in any input, and it is the single fact that decides whether §1 has any path at all	Ask: was it Erase All Content and Settings, a new user account, a reinstall, or a cleaner app? And is the machine Apple silicon or T2
Whether she ever used Save CSV, and where those files went	RP-01 open question 2 already flags that the CSV round-trip may never have been exercised	Ask her; then search every device for <code>lesson-planner-*.csv</code>
Whether a Time Machine or other backup disk exists, and its oldest snapshot date	Not in any input	Ask; then <code>tmutil listbackups</code> on the Mac before any new backup runs

TBD	Why unresolved	What resolves it
Her browser, OS and device, on the old and the current machine	RP-05 called this its highest-value unknown and it is unchanged. It decides which §1 paths exist and how often she must sign in again	One question
The exact Safari <code>localStorage</code> container path on her macOS version	Apple does not document it, and the container layout has moved between releases	The <code>find</code> command in §1 on her machine
The exact Firefox per-origin <code>localStorage</code> filename	Mozilla's own bug describes per-origin database files under QuotaManager storage but the page does not state the filename [S14]	<code>find <profile> -path '*storage/default*' -name '*.sqlite'</code> on her profile
Whether a persistent File System Access grant lets the page write with no gesture in a later session	Chrome documents "indefinite access unless access is revoked" and documents that <code>requestPermission()</code> needs activation, but does not state the combination [S8][S10]	Test on Chrome 122+: store a handle, reload, call <code>queryPermission()</code> then <code>createWritable()</code> with no user gesture
What a local read returns for a pending <code>serverTimestamp()</code> before acknowledgement	The reference page could not be retrieved as prose	Read the <code>SnapshotOptions.serverTimes</code> tamps entry at https://firebase.google.com/docs/reference/js/firestore_
Whether Firestore TTL policies are usable on Spark	The TTL page states no plan requirement, which is an absence rather than a confirmation [S6]	Attempt to enable a TTL policy on a Spark project, or ask Firebase support. Immaterial: §5 uses client-side rotation instead
Real dataset scale	RP-01's open question 7, unchanged: the data is gone	Ask her how many groups and months she kept

Risks specific to this plan

Risk	Severity	Mitigation
A migration bug corrupts good data	High	Migrations are pure functions with unit tests; a snapshot precedes every write; the §7 fingerprint gates acceptance
The human picks the wrong copy during import adjudication	Medium	The losing copy is snapshotted first and the choice is reversible (§3 step 8); the preview shows counts and problems, not just names
Snapshot rotation deletes the wrong snapshots	Medium	Rotation is a pure function of document ids; test it with a fixed id list before it ever runs against real data

Risk	Severity	Mitigation
The restore procedures are never rehearsed	Medium	Run Procedure B end to end once with the teacher, on purpose, while nothing is broken. An unrehearsed restore is a hope
The sync indicator is implemented against the call site	Medium	§4 states the acknowledgement source explicitly; a test must assert the indicator does not advance while offline
Backup files accumulate with a personal IBAN inside	High until the blocker clears	Do not ship Layer 3 to the user before <code>index.html:387-392</code> and <code>index.html:400</code> are fixed

Where an input was wrong

The brief misattributed an RP-05 finding. It states that "RP-05 found Firestore's own docs and changelog contradict each other on the point-in-time-recovery window, and that Free/Spark-tier backups are not downloadable". RP-05 contains no Firestore PITR finding at all. The docs-versus-changelog contradiction is **Supabase's** paused-project restore window — one year on the docs page against 90 days in the changelog (RP-05 §8, sources S12 and S13 there) — and "backups are not available for download" is **Supabase's Free Plan** (RP-05 §4, source S11 there). Following the brief literally would have produced a fabricated Firestore contradiction. The real Firestore position, verified here from scratch: no scheduled backups without Blaze [S1], no PITR without billing [S2], no managed export without Blaze [S3] — which is a worse position than a short PITR window, and is the reason §5 adds a client-side snapshot layer.

A line range in the brief is narrower than it reads. The brief cites the CSV layer at "`index.html:1251-1282`", which covers `serialize` only. `deserialize` runs from 1283 to 1360, and the parser and month normaliser it depends on are at `index.html:1413-1455` and `index.html:1457-1466`. Verified by `grep`.

Nothing else in RP-01 or RP-05 was found to be wrong. Every line reference this report checked — 377-378, 915-931, 935-963, 1187-1213, 1199-1202, 1203-1206, 1208, 1211, 1251-1282 — matched the file.

9. Sources

#	Title	URL	Accessed	Supports
S1	Firestore — Scheduled backups	https://firebase.google.com/docs/firestore/backups	2026-08-20	"This feature requires the Blaze pricing plan"; retention configurable up to 14 weeks; a restore creates a new database rather than replacing the original

#	Title	URL	Accessed	Supports
S2	Firestore — Use point-in-time recovery	https://firebase.google.com/docs/firestore/use-pitr	2026-08-20	Retention is one hour with PITR disabled and seven days when enabled; "Only Google Cloud projects with billing enabled can use the PITR functionality"
S3	Firestore — Export and import data	https://firebase.google.com/docs/firestore/manage-data/export-import	2026-08-20	"Firebase projects must be on the Blaze plan to use the managed export and import service"; exports are written to a Cloud Storage bucket
S4	Firestore — Access data offline	https://firebase.google.com/docs/firestore/manage-data/enable-offline	2026-08-20	Offline persistence is disabled by default on the web and is enabled via persistentLocalCache; on reconnect, multiple changes to the same document resolve last-write-wins
S5	Firestore — Get realtime updates	https://firebase.google.com/docs/firestore/query-data/listen	2026-08-20	Latency compensation: "your listeners will be notified with the new data before the data is sent to the backend"; metadata.hasPendingWrites indicates unsent local changes
S6	Firestore — TTL policies	https://firebase.google.com/docs/firestore/ttl	2026-08-20	Expired data is "typically deleted within 24 hours after its expiration date", deletion order is not guaranteed, and no pricing-plan requirement is stated

#	Title	URL	Accessed	Supports
S7	MDN — <code>Window.showSaveFilePicker()</code>	https://developer.mozilla.org/en-US/docs/Web/API/Window/showSaveFilePicker	2026-08-20	Not Baseline, "does not work in some of the most widely-used browsers"; secure context only; transient user activation required or <code>SecurityError</code> is thrown
S8	MDN — <code>FileSystemHandle.requestPermission()</code>	https://developer.mozilla.org/en-US/docs/Web/API/FileSystemHandle/requestPermission	2026-08-20	Transient user activation is required; limited availability, not Baseline
S9	MDN — File System API	https://developer.mozilla.org/en-US/docs/Web/API/File_System_API	2026-08-20	<code>FileSystemHandle</code> objects "can also be serialized into an IndexedDB database instance", which is what makes a remembered backup file possible at all
S10	Chrome for Developers — Persistent permissions for the File System Access API	https://developer.chrome.com/blog/persistent-permissions-for-the-file-system-access-api	2026-08-20	Chrome 122; three-way prompt with "Allow on every visit" granting "indefinite access unless access is revoked"; installed apps persist permissions automatically; handles stored in IndexedDB

#	Title	URL	Accessed	Supports
S11	MDN — Web Periodic Background Synchronization API	https://developer.mozilla.org/en-US/docs/Web/API/Web_Periodic_Background_Synchronization_API	2026-08-20	Experimental, not Baseline; requires a service worker and a secure context; the user agent decides when the event fires, so no dependable unattended backup
S12	Google — Chrome Sync and your data	https://support.google.com/chrome/a/answer/13616205	2026-08-20	The synced data types are bookmarks, history and tabs, passwords, payment info, settings, extensions and device metadata — website local storage, site data, IndexedDB and cookies are not synced
S13	Chromium — User Data Directory	https://chromium.googlesource.com/chromium/src/+HEAD/docs/user_data_dir.md	2026-08-20	Default user data directory paths on macOS, Windows and Linux, and that each profile is a subdirectory such as Default
S14	Mozilla Bugzilla 1286798 — Next-generation LocalStorage	https://bugzilla.mozilla.org/show_bug.cgi?id=1286798	2026-08-20	Legacy Firefox localStorage is webappsstore.sqlite, "a single database file containing all origins' data"; the current implementation uses per-origin database files under QuotaManager storage

#	Title	URL	Accessed	Supports
S15	Apple — Back up your Mac with Time Machine	https://support.apple.com/en-us/104984	2026-08-20	Hourly backups for the past 24 hours, daily for the past month, weekly for all previous months; the oldest backups are deleted when the backup disk is full
S16	Apple — About Time Machine local snapshots	https://support.apple.com/en-us/102154	2026-08-20	One snapshot of the startup disk approximately every hour, kept for 24 hours, plus one of the last successful backup; snapshots are deleted as they age or as space is needed
S17	Apple — Data Protection in Apple devices	https://support.apple.com/guide/security/data-protection-sece8608431d/web	2026-08-20	Erasing "obliterates all the keys in effaceable storage and renders all user data cryptographically inaccessible"
S18	Apple — Erase Apple devices	https://support.apple.com/guide/deployment/erase-devices-dep0a819891e/web	2026-08-20	The same crypto-erase wording, plus the hardware and OS preconditions for Erase All Content and Settings on Apple silicon and T2 Macs

#	Title	URL	Accessed	Supports
S19	MDN — Storage quotas and eviction criteria	https://developer.mozilla.org/en-US/docs/Web/API/Storage_API/Storage_quotas_and_eviction_criteria	2026-08-20	The 5 MiB per-origin Web Storage cap and eviction behaviour. Inherited from RP-05 source S2, not re-fetched by this report
S20	Firebase — Pricing	https://firebase.google.com/pricing	2026-08-20	Spark plan: 1 GiB stored, 20K document writes, 50K reads and 20K deletes per day, 10 GiB monthly egress, and no payment method required. Re-verified here because §5's snapshot arithmetic depends on it

10. Quick wins

Every item below is new: none duplicates RP-01's six or RP-05's five. Deliberately excluded because they are already claimed elsewhere: the storage try/catch, the versioned JSON backup file, the last-backup age indicator, `navigator.storage.persist()`, and the confirm-before-import dialog. Also excluded as blocked: anything touching the Firestore layer (blocked by RP-05's personal-data blocker) and any change to the CSV parser's leniency (accepted risk in §8).

Rank	Quick win	Effort	Impact	Basis of ranking
1	A raw-dump snippet to run today on every surviving device	XS	Critical	Needs no code change and no deploy, works on corrupt data because it never parses, and it is the only action in this whole report that can still save data that exists right now. Time-sensitive in a way nothing else here is
2	Snapshot the three keys before any destructive action, with an undo	S	Critical	Converts the two verified data-destroying paths (import replace, clear all) from irreversible to reversible. RP-05's confirm dialog prevents the mistake; this one recovers from it, which is strictly stronger
3	Additive schema-version key with a forward-compatibility refusal	S	High	Nothing else in the programme can migrate safely without a version to branch on, and the refusal rule is what stops an old tab silently stripping fields once sync exists. Purely additive to storage

Rank	Quick win	Effort	Impact	Basis of ranking
4	A data fingerprint utility exposed for verification	XS	High	Makes every migration, import and restore in this plan checkable instead of hopeful, and doubles as the integrity check that surfaces RP-01's denormalisation divergence. Read-only by construction
5	Sanitise the currency on import so a bad value cannot brick a group	XS	Medium	RP-01 verified that an imported non-three-letter currency makes a group permanently unopenable with no UI route out. The fix is a guard on the one line that reads the field, on the app's only restore path

PROMPT QW-1: Add a raw storage dump procedure that can be run today on any device
Context: Repo lesson-planner, single deployed file index.html at the repo root, served by GitHub Pages. The app's entire dataset is three localStorage keys: groupLessonPlannerData and groupLessonPlannerSettings, whose names are configured at index.html:377-378, and paymentTemplate, read and written as a bare string literal at index.html:1208 and index.html:1211. RP-01 verified that a corrupt value in groupLessonPlannerData makes App.init() throw, so any exporter that parses the data fails on exactly the data most in need of rescue. The app's existing CSV export at index.html:915-931 both parses and omits the template, so it cannot serve this purpose. Data recovery on any device that still holds data is time-sensitive and must not wait for a deploy.

Task: Create a new file docs/recovery/raw-dump.md containing a short, non-technical, numbered procedure for opening the browser developer console on the app's page and running a single self-contained JavaScript snippet, plus the snippet itself in a fenced javascript block. The snippet must read all three keys with localStorage.getItem and keep the results as raw strings without calling JSON.parse on any of them, wrap them in an object carrying a format identifier, an ISO timestamp and location.origin, and download the result as a .json file using a Blob and an object URL. The procedure must state that it should be run before installing updates, before importing anything, and on every device and browser that may still hold data. Do not modify index.html.

Constraints: Must not modify index.html or any other existing repository file. Must not parse, validate, repair or reformat the stored values in any way; a corrupt value must appear verbatim in the output. Must not delete or write any localStorage key. Must not use any library or any API beyond localStorage, Blob, URL.createObjectURL and a synthetic anchor click, so it runs in Chrome, Safari, Firefox and Edge. Must not include or invite any personal data in the document itself, and must warn that the dumped file contains the payment template and therefore personal payment details, so it must not be emailed or placed in a shared folder. Out of scope: importing the dump back, and any change to the CSV or JSON backup paths.

Acceptance criteria: The new file docs/recovery/raw-dump.md exists and contains exactly one fenced javascript block. Pasting that block into the console on a page with all three keys present downloads a .json file whose parsed content has a raw member with exactly the three key names, whose values are strings, and whose groupLessonPlannerData value is character-identical to the string returned by localStorage.getItem for that key. With groupLessonPlannerData set to the unparseable string [{"name":"Broken"}] the snippet still completes and the downloaded file contains that exact string. With paymentTemplate absent, the downloaded file records null for it rather than omitting the member. No localStorage key is added, changed or removed by running the snippet.

Verification: Serve index.html locally over http://127.0.0.1, seed one group, run the snippet from the console and open the downloaded file to compare the three values against localStorage.getItem output. Repeat with the truncated value planted in groupLessonPlannerData and confirm the download still succeeds. Repeat with paymentTemplate removed. After each run, assert Object.keys(localStorage) is unchanged.

PROMPT QW-2: Snapshot all three storage keys before any destructive action, and add an undo Context: Repo lesson-planner, index.html. Two code paths destroy all existing data. loadFromCsv at index.html:935-963 assigns App.state.groups from the parsed file and persists at index.html:948 with no snapshot and no undo; RP-01 runtime-verified that this silently replaced two real groups, and separately that a CSV containing a balanced stray quote replaced them with a single garbage group. clearAllData at index.html:964-975 confirms and then calls App.services.storage.clear() at index.html:1203-1206, which removes two keys and leaves paymentTemplate behind. There is no way back from either. The storage service is App.services.storage at index.html:1187-1213.

Task: Add a snapshot function to App.services.storage that copies the current raw string values of all three keys groupLessonPlannerData, groupLessonPlannerSettings and paymentTemplate into keys named by appending .snapshot.<epochMilliseconds> to each original key name, recording absent keys as absent rather than as empty strings. Call it immediately before the state assignment in loadFromCsv and immediately before storage.clear() in clearAllData. Add a matching restore function that writes a chosen snapshot generation back over the three keys and re-renders, and expose it through a toolbar control that is visible only when at least one snapshot generation exists, labelled to say it undoes the last replacement. Keep at most the three most recent generations, deleting older ones in the same pass.

Constraints: Do not change the three localStorage key names or the persisted data shape of any of them; snapshot keys are additive and no other code may read them. Do not change the parsing, validation or error messages of the CSV importer, and do not change the Clear All Data confirmation text. Do not add snapshot keys to storage.clear(). Snapshots must be taken from raw getItem values without parsing, so a corrupt value is preserved verbatim. Use the confirm mechanism already used by Clear All Data rather than a custom modal. No new dependencies, no build step. Out of scope: the JSON backup file format, and cloud sync.

Acceptance criteria: With two groups, a non-default currency and a customised template present, importing a CSV creates keys groupLessonPlannerData.snapshot.<n>, groupLessonPlannerSettings.snapshot.<n> and paymentTemplate.snapshot.<n> for the same <n>, and the undo control then restores all three to their pre-import values exactly, including the template. Clear All Data also creates a snapshot generation, and undo after it restores the two cleared keys. Performing four destructive actions leaves exactly three snapshot generations. When paymentTemplate is absent before the action, restoring that generation leaves it absent rather than setting it to an empty string. When no snapshot exists the undo control is not shown. Importing a CSV that throws leaves the pre-existing data intact and does not leave a partial snapshot generation behind.

Verification: Serve locally, build the two-group fixture with a custom template, export a CSV, import a different CSV, then read all snapshot keys from the console and assert they match values captured before the import. Click undo and assert the three live keys are byte-identical to the captured values and the group list and Edit Template contents match. Repeat for Clear All Data. Run four destructive actions and assert the count of keys matching .snapshot. is exactly nine, being three generations of three keys.

PROMPT QW-3: Add an additive schema-version key and refuse to write when the stored version is newer

Context: Repo lesson-planner, index.html. RP-01 verified that none of the three storage keys carries a schema version, so no future migration has anything to branch on and no client can tell whether stored data was written by an older or a newer build.

App.services.storage.load() at index.html:1188-1198 assigns parsed values straight into App.state, and save() at index.html:1199-1202 writes both config keys unconditionally. A durable-storage plan for this app writes the whole state as one document, so a client that does not understand a newer shape will silently delete the fields it does not know about the first time it saves.

Task: Introduce a module-level constant holding the current schema version as the integer 1, and a new additive localStorage key named groupLessonPlannerSchemaVersion holding that number as a decimal string. In load(), read the key, treat an absent or unparseable value as version

0, and treat 0 and 1 as loadable. If the stored value parses to an integer greater than the constant, do not assign anything into App.state, set a new App.state flag named readOnlyReason to a short string naming the version found and the version expected, and render a non-blocking message telling the user the data was saved by a newer version and to reload the page. In save(), return without writing anything when readOnlyReason is set. When loading succeeds and the key was absent, write the constant to the key on the next successful save rather than during load.

Constraints: Do not change the three localStorage key names groupLessonPlannerData, groupLessonPlannerSettings and paymentTemplate, and do not change the persisted data shape of any of them. Do not write the version key during load(); load must remain free of side effects on storage. Do not implement any migration function in this prompt; version 0 and version 1 describe the same shape. Do not block rendering and do not use alert or confirm for the newer-version case. No new dependencies, no build step. Out of scope: the Firestore document, the JSON backup envelope, and any repair of malformed data.

Acceptance criteria: On a profile with no version key, the app loads normally, and after any action that calls save() the key groupLessonPlannerSchemaVersion holds exactly the string 1 while the other three keys are byte-identical in shape to before the change. With the key set to 1, behaviour is identical to today. With the key set to 2, the group list does not render stored groups, a visible message mentions a newer version, App.state.readOnlyReason is a non-empty string, and after attempting an edit the three data keys are byte-identical to their pre-attempt values. With the key set to the string abc, the app loads normally and treats it as version 0. No uncaught error appears in the console in any of these cases.

Verification: Serve locally over http://127.0.0.1. For each of the four key states above, set the value from the console, reload, and assert the rendered group count, the presence or absence of the message, the value of App.state.readOnlyReason via page.evaluate, and an empty console error list. In the version 2 case, capture all three data keys before and after clicking Add Group and Save and assert they are unchanged. In the absent case, add a group and assert the version key now reads 1.

PROMPT QW-4: Expose a data fingerprint utility for verifying imports, restores and migrations Context: Repo lesson-planner, index.html. App.utils at index.html:1365 already holds pure helpers such as iso at index.html:1394-1396 and toMonthKey at index.html:1398-1401. There is no way to compare the dataset before and after a destructive operation, so an import, a restore or a future migration can only be checked by eye. RP-01 also verified a latent invariant problem: every date is stored twice, once in group.dates and once in monthlyOverrides[m].dates, synchronised only in saveDateChanges at index.html:745-772, so the two totals can diverge silently. JSON.stringify preserves insertion order, so a naive checksum of two semantically identical datasets can differ.

Task: Add a pure function App.utils.dataFingerprint() that reads the three localStorage keys, parses the two JSON ones defensively so a corrupt value yields a reported problem rather than a throw, and returns an object with the group count, the total of group.dates lengths, the total of monthlyOverrides dates lengths, a boolean stating whether those two totals are equal, the number of distinct month keys, the character length of the payment template or null when the key is absent, an array of problem strings listing every date key not matching four digits hyphen two digits hyphen two digits, every month key not matching four digits hyphen two digits, and every group whose currency is not exactly three ASCII letters, and a short hexadecimal checksum computed over a canonical form in which object keys are sorted and array members are sorted so the result is independent of insertion order.

Constraints: Must be read-only: it must not write, remove or reorder any localStorage key, and must not mutate App.state. Do not change the three localStorage key names or the persisted data shape. Do not write the canonical form back to storage anywhere. Do not surface it in the user interface in this prompt; it is a developer and verification tool reachable from the console. Do not add a dependency and do not use any hashing library; an inline non-cryptographic checksum is sufficient and must be deterministic across browsers. No build step. Out of scope: repairing any problem it reports.

Acceptance criteria: With a dataset of two groups, six month keys and forty-one dates present

in both places, the returned object reports 2, 41, 41, true and 6, an empty problems array, and a checksum string. Reloading the page and calling it again returns an identical checksum. Reordering the members of one group's dates array in storage and reloading returns the same checksum. Adding one date changes the checksum. Planting a date key of 5-08-10 adds exactly one problem entry naming that value, and planting a currency of US Dollar adds exactly one problem entry naming that group. With groupLessonPlannerData set to an unparseable string the function returns a problems array describing it and does not throw. Calling it never changes Object.keys(localStorage) or any stored value.

Verification: Serve locally over http://127.0.0.1, build the fixture, and call App.utils.dataFingerprint() via page.evaluate, asserting each field. Capture all localStorage values before and after the call and assert they are identical. Reorder a dates array from the console, reload, and assert the checksum is unchanged. Then add a date and assert it changed. Finally plant the malformed date, the malformed currency and the unparseable value in turn and assert the problems array in each case.

PROMPT QW-5: Sanitise the currency during CSV import so a bad value cannot brick a group
Context: Repo lesson-planner, index.html. App.services.csv.deserialize at index.html:1283-1360 accepts any non-empty string as a group's currency at index.html:1317.

App.utils.formatCurrency uses Intl.NumberFormat, which accepts any three ASCII letters but throws RangeError otherwise. RP-01 runtime-verified the consequence: importing a currency of US Dollar makes the group permanently unopenable, because the throw happens inside render.monthlyOverrides() which runs before the modal is shown, so openGroupModal aborts and there is no UI route to open, fix or delete that group; only Clear All Data or another import escapes. The currency select at index.html:291-294 offers only UAH and PLN, and the app's fallback currency elsewhere is UAH.

Task: In deserialize, normalise each row's currency value before it reaches the group object: trim it, uppercase it, and accept it only when it matches exactly three ASCII letters. When it does not match, fall back to the existing default currency for that import and record the substitution. After the import completes, if any substitution was made, show a single non-blocking message naming how many groups had their currency replaced and what value was used, so the user is told rather than surprised later.

Constraints: Do not change the CSV column names, order or quoting, and do not change any existing error message, including the Unable to load CSV prefix used at index.html:951. Do not reject the whole import because of a bad currency; a single bad cell must never abort a restore. Do not change the three localStorage key names groupLessonPlannerData, groupLessonPlannerSettings and paymentTemplate, and do not change the persisted data shape. Do not add validation to the currency select, and do not attempt to repair currencies already stored in localStorage; this prompt guards the import path only. Do not use alert for the substitution notice. No new dependencies, no build step. Out of scope: whether a three-letter but meaningless code such as XYZ should be rejected, which remains accepted because Intl formats it without error.

Acceptance criteria: Importing a CSV whose Currency cell is US Dollar produces a group that opens normally, with a currency of exactly three uppercase letters, and a visible notice reporting one substitution. Importing a CSV whose Currency cell is pln produces a group with currency PLN. Importing a CSV whose Currency cell is empty produces a group whose currency is the import default rather than an empty string. Importing a well-formed CSV exported by the app produces byte-identical stored data to before this change. A CSV that previously threw for a missing column or an invalid month still throws the same message and still leaves existing data untouched.

Verification: Serve locally, create a group, export a CSV, then hand-edit copies whose Currency cell contains US Dollar, pln and an empty value. Import each and assert the group card opens, that App.state.groups[0].currency matches three uppercase letters, and that the notice appears only for the substitution cases. Re-import the unmodified export and assert groupLessonPlannerData is byte-identical to the value captured before the change. Finally import a CSV with a missing Name column and assert the original error message and that the existing data is unchanged.